TRABAJO PRÁCTICO INTEGRADOR

Segunda entrega · Unidades 4 a 8

Proyecto: Food Store

Base de Datos II · Tecnicatura Universitaria en Programación

Universidad Tecnológica Nacional · FRM

10 de noviembre de 2026

Integrantes

Benjamin Lagos, Leandro Rea, Tomas Martin, Brian Rampone, Santiago Prandina, Emir Gris, Joel Herrera y Octavio Medina.

Repositorio del proyecto

https://github.com/Tomimartin/BaseDeDatos2

Este informe reúne los TP de las unidades 4 a 8. Partimos de los scripts, informes, consultas y capturas que ya habíamos presentado. Para el integrador agregamos las pruebas de reversión de la Unidad 4, realizadas en una copia restaurada. Los documentos de Kiro y ambas capturas ya están subidos al repositorio.

La mayor parte de las pruebas se conserva en las carpetas de cada unidad. Algunas consultas auxiliares de la Unidad 4, como la carga masiva y la creación del índice del reporte, se ejecutaron por separado y no quedaron incluidas en el script publicado de desnormalización. No confundimos esa diferencia de documentación con una tarea que no se haya realizado.

1. Introducción

En Food Store fuimos trabajando distintos aspectos de las bases de datos: primero el diseño de las tablas, después los permisos y la auditoría, las copias de seguridad, MongoDB y finalmente una base SQLite para trabajar sin conexión. En este informe juntamos esas partes y explicamos qué hicimos, cómo lo probamos y por qué elegimos cada solución.

Trabajamos principalmente con PostgreSQL 17.11. En la Unidad 7 usamos MongoDB 8.0.10 para representar pedidos como documentos y, en la Unidad 8, SQLite para simular el funcionamiento de una tablet. No reemplazamos PostgreSQL: seguimos usando esa base para la información principal del negocio, como pedidos, precios y stock.

Para revisar el trabajo usamos los scripts, resultados de consultas, capturas y logs conservados. También nos basamos en los apuntes de la cátedra, aplicándolos al caso Food Store. El repositorio se revisó nuevamente para esta versión del informe.

2. Unidad 4. Normalización y desnormalización

2.1. Normalización a FNBC

En esta unidad partimos de la tabla control_lote_almacen, que relaciona un lote, un depósito y el responsable del control. Según la regla del ejercicio, un lote y un depósito determinan al

responsable, y cada responsable pertenece a un solo depósito. Esto se expresa con las dependencias {L,D} → R y R → D.

El problema era que R → D no cumple la Forma Normal de Boyce-Codd: conocer al responsable nos dice el depósito, pero no identifica un lote concreto. Por eso separamos los datos en responsable_deposito y control_lote. Después usamos la vista control_lote_almacen_compat para volver a obtener la misma información mediante una unión.

Comprobamos que la descomposición no perdiera datos: teníamos 3 filas en la tabla original, reconstruimos las mismas 3 y la comparación con EXCEPT devolvió 0 diferencias. Tampoco aparecieron claves duplicadas. También consultamos si algún responsable estaba en más de un depósito y no encontramos casos en los registros del ejercicio.

Hay una limitación que dejamos anotada: con las claves primarias y foráneas de las tablas nuevas no alcanza para impedir cualquier futura violación de {L,D} → R. Si se usara en un sistema real con nuevas altas, haría falta un control adicional entre tablas.

2.2. Prueba de reversión para el integrador

El TP4 original ya estaba realizado, pero el integrador pedía además especificar la migración en Kiro y contar con un plan de reversión. Preparamos requirements.md, design.md y tasks.md, que ya están publicados en TP4_unidad4/docs/kiro/. Para las pruebas usamos food_store_copia, una base restaurada, y creamos el esquema separado tpi_u4_rollback_lab. También subimos las capturas 07 y 08 de las reversiones.

Prueba	Resultado que obtuvimos
Migración a las dos tablas	3 filas reconstruidas; 0 diferencias y 0 duplicados
BEGIN y ROLLBACK	Quedaron las 3 filas originales; tablas nuevas y vista en NULL
Migración con COMMIT	2 responsables, 3 controles y 3 filas reconstruidas
Reversión después del COMMIT	DROP RESTRICT sobre los objetos del laboratorio; 3 filas originales intactas

La última prueba muestra que pudimos revertir esta migración particular después de confirmarla. Los DROP RESTRICT afectaron únicamente los objetos creados en el laboratorio de food_store_copia; no ejecutamos esos DROP en la base original foodstorecopia ni usamos CASCADE.

2.3. Desnormalización del reporte

En la segunda parte del TP queríamos obtener las cinco categorías con mayores ventas del día. La consulta original unía detalle_pedido, pedido, producto y categoria. Trabajamos con una copia de datos más poblada y probamos una alternativa que guarda también en detalle_pedido la fecha_venta, la categoria_venta y el indicador pedido_eliminado.

Creamos funciones y triggers para que esos campos se mantuvieran actualizados cuando cambiaban los datos de origen. También creamos y probamos un índice parcial para el reporte, comparamos los planes de ejecución con EXPLAIN ANALYZE y usamos una consulta de auditoría para buscar inconsistencias. Esta opción permite evitar varias uniones al consultar el reporte, aunque guardar datos repetidos consume espacio y puede exigir más trabajo al actualizar.

La carga masiva y el índice parcial de la Unidad 4 se ejecutaron durante el TP, según los registros del equipo, aunque el archivo publicado tp_desnormalizacion_top_categorias.sql reúne principalmente las columnas calculadas, los cuatro triggers, la consulta del reporte con EXPLAIN ANALYZE y la auditoría. Las capturas de los planes y el control de consistencia están en TP4_unidad4/imagenes/.

3. Unidad 5. Seguridad y auditoría

En esta unidad dejamos de trabajar solamente con las tablas y nos enfocamos en quién puede consultar o modificar cada dato. Aplicamos el principio de mínimo privilegio: cada rol debe poder hacer únicamente lo que necesita. Para eso creamos roles de grupo de lectura, escritura, soporte y reportes, y cuentas con LOGIN como app_web y admin_datos.

Después probamos los permisos reales. Por ejemplo, la aplicación podía realizar ciertas operaciones con pedidos, pero no borrar físicamente registros ni cambiar columnas que debían estar protegidas, como el precio unitario histórico. Corregimos permisos por columna para que la protección no dependiera solamente del código de la aplicación.

El trabajo de seguridad también incluyó la configuración del logging, la lectura de registros del servidor y una comparación de resultados con datos anonimizados. Los scripts, informes y capturas están separados en las partes A, B, C y D de TP5_unidad5.

También hicimos una simulación de incidente. En una función de prueba hubo tres intentos de autenticación fallidos y otro que devolvió verdadero en la ejecución registrada. Luego se intentó leer información sin permiso y asignar admin_datos a app_web. Ambos intentos fueron rechazados con el código 42501.

Usamos los registros de auditoría y OpenCode para reconstruir la secuencia y revisamos sus conclusiones contra las capturas. La cuarta llamada devolvió true en la captura, aunque el log SQL por sí solo no permite conocer el valor retornado. Fue una simulación controlada, no un ataque real: no hay evidencia de extracción de datos ni de compromiso de la base.

4. Unidad 6. Respaldos, mantenimiento y recuperación

El trabajo de la Unidad 6 consistió en comprobar que una copia de seguridad realmente se pudiera usar. No alcanzaba con guardar un archivo: había que restaurarlo en otra base y revisar que los datos coincidieran.

El 6 de noviembre hicimos un respaldo de foodstorecopia con pg_dump en formato custom. El archivo ocupó 7.053.279 bytes. Después lo restauramos en foodstore_restore_test usando pg_restore. Los conteos de las cinco tablas coincidieron y los controles de productos y detalles inválidos dieron cero. Además, ejecutamos ANALYZE VERBOSE y revisamos estadísticas y conexiones del servidor.

Dato del ensayo	Resultado
Tiempo de recuperación (RTO)	4,673 segundos; objetivo: 300 segundos
Antigüedad del respaldo usada como referencia de RPO	13,309 minutos; referencia: 24 horas
Categorías / clientes / productos	2.006 / 20.004 / 50.010
Pedidos / detalles	200.004 / 200.014
Registros inválidos detectados	0

El TP6 se documentó con los archivos de respaldo, los registros de restauración, las comparaciones de conteos y las pruebas de mantenimiento. SPEC_U6.md también fija frecuencias sugeridas para las tareas periódicas. Este informe describe los ensayos efectivamente registrados y no presenta esas frecuencias como ejecuciones automáticas ya comprobadas.

5. Unidad 7. MongoDB y comparación con PostgreSQL

En la Unidad 7 quisimos ver cómo quedaban algunos pedidos si, en vez de consultar varias tablas relacionadas, los guardábamos en documentos. Para eso elegimos cinco pedidos reales de PostgreSQL: 180991, 181008, 181019, 181036 y 181045.

Creamos la colección pedidos_u7 en MongoDB con datos de cliente representados de forma anónima y los productos embebidos dentro de cada pedido. Conservamos cantidades, precios históricos y fechas; utilizamos Decimal128 para evitar pérdidas de precisión en importes. Después hicimos consultas sobre los mismos cinco pedidos, por estado, producto y total, y comparamos sus identificadores con PostgreSQL.

En MongoDB los cinco pedidos se guardaron como documentos que contienen cliente e ítems. El origen fue una exportación CSV de PostgreSQL; el repositorio conserva el CSV, las consultas de ambos motores y las salidas del ensayo. No se modificó el stock desde MongoDB.

Los resultados observados fueron cinco documentos en total, tres pedidos activos, uno con el producto 30998 y cuatro con un total superior a 10.000. Las comparaciones de estado y producto coincidieron con PostgreSQL; no encontramos ítems inválidos ni totales inconsistentes en los cinco documentos. MongoDB es práctico para leer pedidos completos, pero no medimos una ventaja de tiempo frente a PostgreSQL. Por las relaciones, las restricciones y el control del stock, mantuvimos PostgreSQL como fuente principal del negocio.

6. Unidad 8. SQLite y funcionamiento sin conexión

La idea de esta unidad fue simular lo que haría Food Store si un vendedor tuviera una tablet sin Internet. La tablet tiene que poder mostrar productos y guardar borradores, pero no puede decidir por su cuenta que una venta está confirmada.

Preparamos foodstore_movil.db en SQLite con cuatro tablas principales: producto_cache, pedido_borrador, item_borrador y cola_sync. Importamos un catálogo local de 50.010 productos. Aplicamos claves primarias y foráneas, CHECK y STRICT, y expresamos los precios en centavos enteros. Probamos la atomicidad de las operaciones con COMMIT y ROLLBACK sobre una copia de la base.

Después armamos un semáforo para evaluar solicitudes pendientes contra una instantánea del catálogo del servidor. Los resultados podían ser RECHAZADO, CONFLICTO o SIMULADO_APROBADO. Este último estado es solo una comparación favorable: no confirma una venta ni descuenta stock en PostgreSQL. Los casos C1 a C7 probaron catálogo sin conexión, cambios de precio, falta de stock, bajas, reenvíos y restricciones de integridad.

Caso probado	Resultado
C1. Consultar el catálogo	50.010 productos en la base local
C2. Cambió el precio	CONFLICTO; actualizado el borrador, aprobación simulada
C3. No hay stock suficiente	RECHAZADO
C4. Producto dado de baja	RECHAZADO
C5. Datos coincidentes	SIMULADO_APROBADO
C6. Repetir un UUID	La PK rechaza el duplicado (1555)
C7. Cantidad o referencia incorrecta	CHECK / FK rechazan los datos (275 / 787)

En esta unidad usamos Kiro para revisar requirements.md, design.md y tasks.md. Con OpenCode Plan analizamos scripts y contrastamos tres hallazgos; con OpenCode Build generamos alternativas y comprobamos cambios sobre copias. No aceptamos todo lo propuesto: conservamos

el índice ux_cola_una_pendiente, porque impide dos solicitudes pendientes del mismo borrador, algo distinto de la PK que evita UUID duplicados. También corregimos el tratamiento de NULL para que los valores desconocidos no terminen como una aprobación simulada.

7. Cómo se relacionan las tres bases

Después de hacer las prácticas, nos quedó claro que las tres bases no cumplen la misma función. PostgreSQL conserva los datos oficiales y aplica las reglas del negocio. MongoDB nos sirvió para probar otra forma de consultar pedidos completos. SQLite permite trabajar con información local mientras no hay conexión.

Por eso mantuvimos una regla simple para la simulación móvil: la tablet prepara el pedido, pero el servidor es quien decide si puede concretarse. Así evitamos considerar como venta real una solicitud basada en precios o stock viejos.

8. Herramientas de IA y nuestras decisiones

Utilizamos Kiro para preparar o revisar especificaciones, OpenCode para la auditoría de permisos y el trabajo de SQLite, y Codex para los scripts de respaldo y restauración de la Unidad 6. La asistencia conversacional se utilizó para comprender consignas, revisar prompts y consultas, discutir errores y mejorar la documentación. En cada caso se contrastaron las propuestas con los resultados de las pruebas.

Hubo recomendaciones que no seguimos. En seguridad ajustamos los permisos en lugar de dar accesos amplios. En MongoDB no afirmamos que fuera más rápido sin medirlo. En SQLite mantuvimos un índice que protegía una regla distinta a la clave primaria y corregimos el manejo de NULL. Anotamos esas decisiones y las pruebas en las bitácoras de cada unidad.

Los pedidos, hallazgos y decisiones de IA están registrados en las bitácoras de las unidades. Para esta segunda entrega agregamos una DUIA general en Markdown y una revisión consolidada de scripts como anexos.

9. Relación de evidencias con los 14 puntos del integrador

Para que la defensa sea más fácil, vinculamos cada objetivo con la evidencia o documento correspondiente. Esta tabla resume lo presentado; no reemplaza la comprobación de cada condición de la consigna.

Punto	Qué tenemos	Documento o evidencia
1	Dependencias, descomposición y EXCEPT en TP4_unidad4	TP4 / scripts y captura 01
2	Especificación Kiro; ROLLBACK y reversión post-COMMIT	Kiro U4 / capturas 07 y 08
3	Reporte, triggers, EXPLAIN ANALYZE, índice y auditoría	TP4 / scripts y capturas 02-06
4	Roles mínimos, permisos por columna y verificaciones	U5 / Parte A
5	Simulación de acceso indebido y análisis de logs	U5 / Partes B y C
6	Análisis asistido por IA y decisiones sobre permisos	U5 / bitácora y auditoría

Punto	Qué tenemos	Documento o evidencia
7	Respaldo, restauración, estadísticas y plan periódico	U6 / SPEC, scripts y registros
8	Restauración cronometrada y validaciones posteriores	U6 / informe y log de restore
9	Pedidos reales migrados a MongoDB	U7 / CSV, JS y resultados
10	Comparación de consultas y justificación de elección	U7 / informe y SQL
11	Base SQLite y especificación con Kiro; revisión Build	TP8 / schema, Kiro y bitácora
12	Simulación de conflictos C1 a C7 y pruebas de integridad	TP8 / SQL, DB y evidencias
13	Revisión OpenCode Plan, hallazgos y lectura estática adicional	TP8 / bitácora; anexo técnico
14	Capa de datos y documentación integrada	GitHub / informe y DUIA anexa

10. Conclusión

Los TP de las cinco unidades nos permitieron trabajar sobre problemas diferentes del mismo proyecto: normalización y reportes, permisos y auditoría, recuperación ante fallas, consultas documentales y pedidos sin conexión. En cada unidad analizamos los resultados a partir de consultas, scripts o capturas y dejamos asentadas las decisiones tomadas.

Para el integrador agregamos a la Unidad 4 dos pruebas de reversión en un esquema separado de una base restaurada: una antes de confirmar los cambios y otra después del COMMIT. En el resto de las unidades reutilizamos los materiales y resultados de los TP originales.

Los documentos de Kiro y las capturas 07 y 08 de la Unidad 4 ya están en GitHub. Junto con este informe dejamos una DUIA general y una revisión técnica consolidada. La sección de reversión del TP4 incorpora referencias a ambas imágenes en la versión lista para publicar.

11. Material consultado y archivos del proyecto

Apuntes de la cátedra: Unidad 4 (normalización avanzada y esquemas heredados); Unidad 5 (roles y auditoría); Unidad 6 (mantenimiento); Unidad 7 (NoSQL); Unidad 8 (bases móviles e integración).

Repositorio revisado: github.com/Tomimartin/BaseDeDatos2, rama main. Carpetas consultadas: TP4_unidad4, TP5_unidad5, U6, U7 y TP8.

Fuentes principales: TP4_unidad4/TP4_unidad4.md y docs/kiro/tasks.md; TP5_unidad5/Parte_A a Parte_D; U6/INFORME_U6.md y SPEC_U6.md; U7/tp_Unidad7.md; TP8/README.md, docs/bitacora_ia.md y docs/informe.pdf.

El repositorio contiene los scripts y la documentación principal de U4 a U8. El respaldo binario de U6 se conserva en el equipo del ensayo, acompañado por logs y comprobaciones. El SQL auxiliar de algunas pruebas del reporte de U4 no está reunido en el archivo SQL principal; los resultados y las capturas del TP se conservaron por separado.

Anexo. Capturas de las pruebas

Incluimos una selección de capturas. Las demás están guardadas en las carpetas de evidencias de cada unidad.

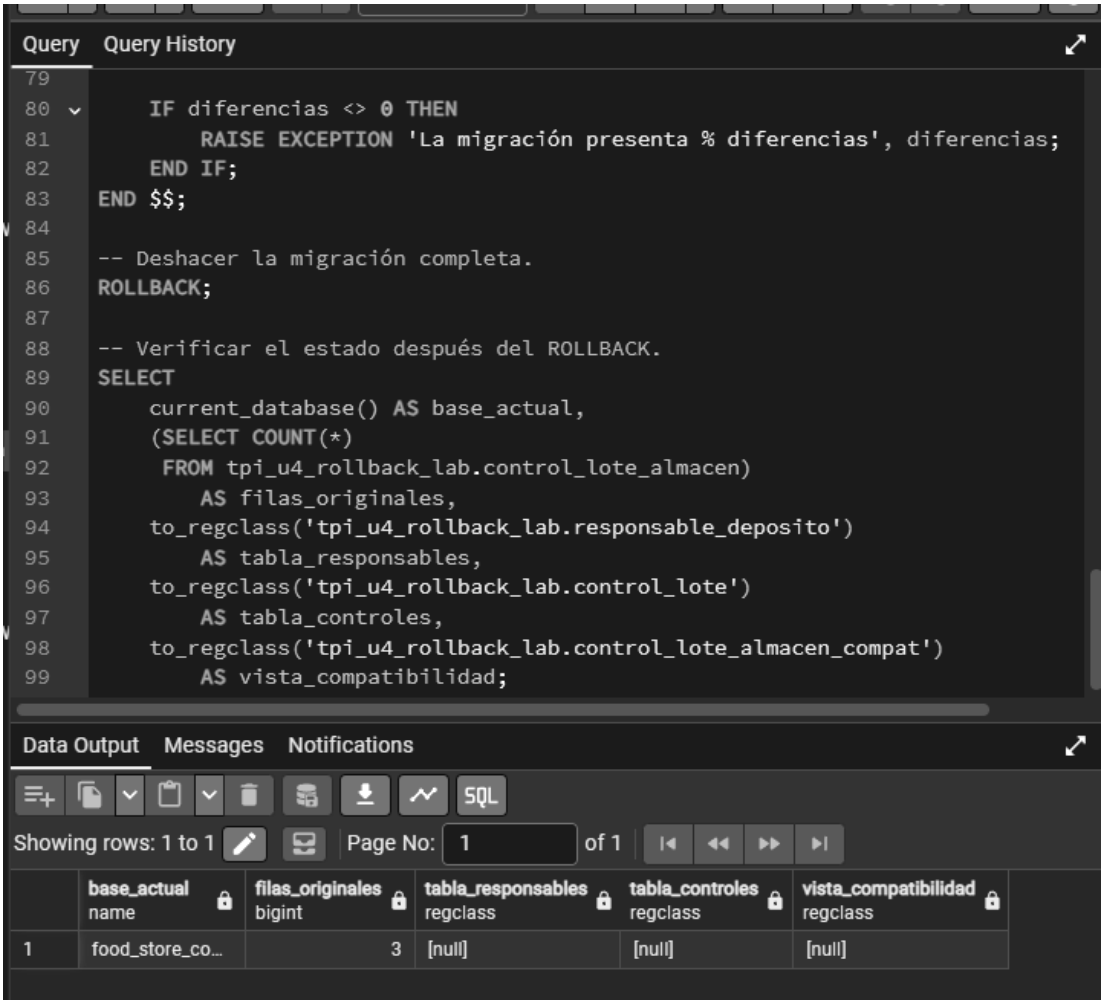


Figura 1. Unidad 4: después del ROLLBACK quedan las tres filas originales y no existen las tablas nuevas.

Query | Query History

```
-- RESTRICT impide eliminar objetos con otras dependencias.
DROP VIEW tpi_u4_rollback_lab.control_lote_almacen_compat RESTRICT;

DROP TABLE tpi_u4_rollback_lab.control_lote RESTRICT;

DROP TABLE tpi_u4_rollback_lab.responsable_deposito RESTRICT;

-- 3. Confirmar los cambios.
COMMIT;

-- 4. Comprobar el resultado final.
SELECT
    current_database() AS base_actual,
    (SELECT COUNT(*)
     FROM tpi_u4_rollback_lab.control_lote_almacen)
        AS filas_originales,
    to_regclass('tpi_u4_rollback_lab.responsable_deposito')
        AS tabla_responsables,
    to_regclass('tpi_u4_rollback_lab.control_lote')
        AS tabla_controles,
    to_regclass('tpi_u4_rollback_lab.control_lote_almacen_compat')
        AS vista_compatibilidad;
```

Data Output | Messages | Notifications

Showing rows: 1 to 1 | Page No: 1 of 1

	base_actual name	filas_originales bigint	tabla_responsables regclass	tabla_controles regclass	vista_compatibilidad regclass
1	food_store_co...	3	[null]	[null]	[null]

Figura 2. Unidad 4: reversión posterior al COMMIT en el esquema de prueba.

```
GRANT admin_datos TO app_web;
```

Data Output | Messages | Notifications

```
ERROR:  se ha denegado el permiso para otorgar el rol «admin_datos»
Sólo los roles con la opción ADMIN en el rol «admin_datos» pueden otorgar este rol.

SQL state: 42501
Detail: Sólo los roles con la opción ADMIN en el rol «admin_datos» pueden otorgar este rol.
```

Figura 3. Unidad 5: intento de elevar permisos rechazado.

```
PS C:\Windows\System32> powershell.exe -NoProfile -ExecutionPolicy Bypass -File "C:\Users\ignac\Downloads\restore
s1" -PgBin "C:\Program Files\PostgreSQL\17\bin" -BackupFile "C:\Users\ignac\backups\foodstorecopia_20261106_14511
ump" -RecreateTestDatabase
[2026-11-06T15:04:23.3154928-03:00] PREPARACION origen_del_respaldo=foodstorecopia destino=foodstore_restore_test
o=C:\Users\ignac\backups\foodstorecopia_20261106_145119_926.dump
La opcion RecreateTestDatabase permite eliminar y recrear solo foodstore_restore_test.
Ingrese la contrasena de PostgreSQL. Se usa solo durante este proceso y no se guarda en archivos.
Contrasena: ***************
[2026-11-06T15:04:38.4281309-03:00] PREPARACION eliminando exclusivamente foodstore_restore_test
[2026-11-06T15:04:38.6557994-03:00] RTO previsto: 300 segundos. RPO previsto: 24 horas.
[2026-11-06T15:04:38.6578100-03:00] El cronometro incluye crear la base, restaurar, consultar y comparar las vali
s.

tabla          antes_del_respaldo despues_de_restaurar coincide
-----          ------------------ -------------------- --------
categoria                    2006                 2006     True
cliente                     20004                20004     True
producto                    50010                50010     True
pedido                     200004               200004     True
detalle_pedido             200014               200014     True


[2026-11-06T15:04:43.3575193-03:00] EXITO restauracion y validaciones completas en foodstore_restore_test
[2026-11-06T15:04:43.3605200-03:00] RTO observado: 4,673 segundos; objetivo: 300 segundos
[2026-11-06T15:04:43.3625194-03:00] RPO observado: 13,309 minutos de antiguedad; objetivo: 1440 minutos
[2026-11-06T15:04:43.3645185-03:00] Productos invalidos=0; detalles invalidos=0
[2026-11-06T15:04:43.3657360-03:00] RESULTADO C:\Users\ignac\logs\resultado_restore_20261106_150423_239.json
[2026-11-06T15:04:43.3687377-03:00] COMPARACION C:\Users\ignac\logs\comparacion_20261106_150423_239.csv
PS C:\Windows\System32>
```

Figura 4. Unidad 6: comparación de datos luego de la restauración.

Query | Query History

```
26  SELECT
27      'Producto 30998' AS consulta,
28      COUNT(*) AS cantidad,
29      COALESCE(
30          array_agg(p.id_pedido ORDER BY p.id_pedido),
31          '{}'::bigint[]
32      ) AS ids
33  FROM pedidos_migrados p
34  WHERE EXISTS (
35      SELECT 1
36      FROM public.detalle_pedido dp
37      WHERE dp.id_pedido = p.id_pedido
38        AND dp.id_producto = 30998
39  )
40  ORDER BY consulta;
```

Data Output | Messages | Notifications

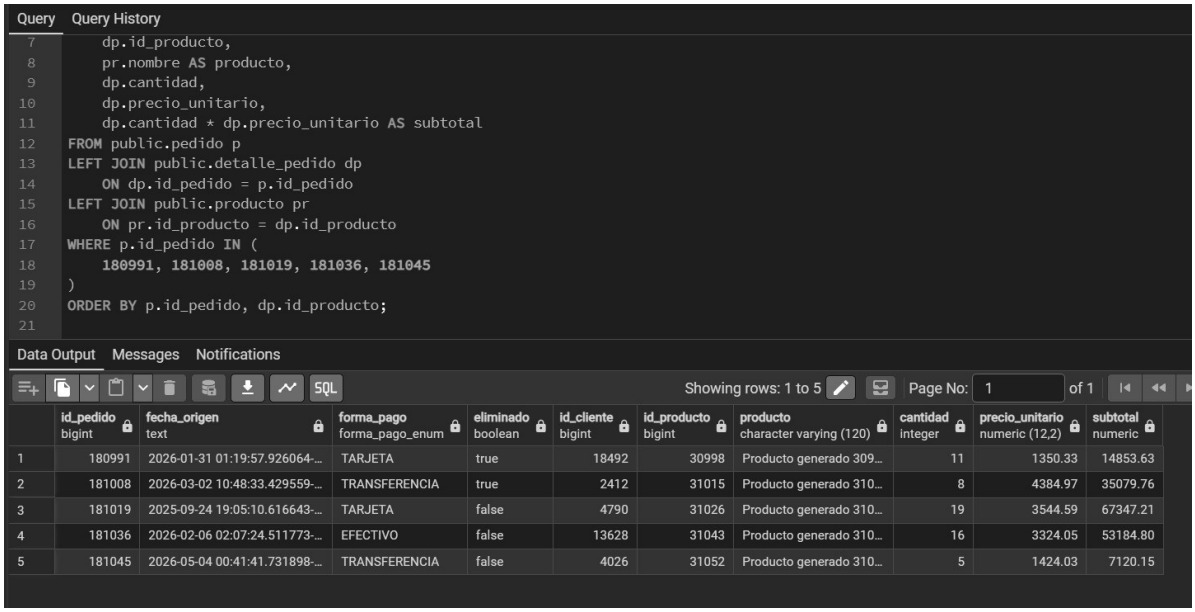
SQL

	consulta text	cantidad bigint	ids bigint[]
1	Estado ACTIVO	3	{181019,181036,181045}
2	Producto 309…	1	{180991}

Figura 5. Unidad 7: resultados utilizados para comparar con MongoDB.

Evidencias adicionales: unidades 7 y 8

Las imágenes siguientes corresponden a consultas y pruebas realizadas en PostgreSQL y SQLite. Se conservan los resultados originales; solo se ajustó el color para mantener el formato del informe.



Query | Query History

```
    dp.id_producto,
    pr.nombre AS producto,
    dp.cantidad,
    dp.precio_unitario,
    dp.cantidad * dp.precio_unitario AS subtotal
FROM public.pedido p
LEFT JOIN public.detalle_pedido dp
    ON dp.id_pedido = p.id_pedido
LEFT JOIN public.producto pr
    ON pr.id_producto = dp.id_producto
WHERE p.id_pedido IN (
    180991, 181008, 181019, 181036, 181045
)
ORDER BY p.id_pedido, dp.id_producto;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1 of 1

	id_pedido bigint	fecha_origen text	forma_pago forma_pago_enum	eliminado boolean	id_cliente bigint	id_producto bigint	producto character varying (120)	cantidad integer	precio_unitario numeric (12,2)	subtotal numeric
1	180991	2026-01-31 01:19:57.926064-...	TARJETA	true	18492	30998	Producto generado 309...	11	1350.33	14853.63
2	181008	2026-03-02 10:48:33.429559-...	TRANSFERENCIA	true	2412	31015	Producto generado 310...	8	4384.97	35079.76
3	181019	2025-09-24 19:05:10.616643-...	TARJETA	false	4790	31026	Producto generado 310...	19	3544.59	67347.21
4	181036	2026-02-06 02:07:24.511773-...	EFECTIVO	false	13628	31043	Producto generado 310...	16	3324.05	53184.80
5	181045	2026-05-04 00:41:41.731898-...	TRANSFERENCIA	false	4026	31052	Producto generado 310...	5	1424.03	7120.15

Figura 6. Unidad 7: los cinco pedidos originales de PostgreSQL utilizados para la migración a MongoDB.



SQL 1*

```
SELECT
    solicitud_uuid,
    estado,
    motivo
FROM cola_sync
ORDER BY creada_en, solicitud_uuid;
```

	solicitud_uuid	estado	motivo
1	bbbbbbbb-0000-4000-8000-000000000001	SIMULADO_APROBADO	Datos coinciden con la instantanea
2	dddddddd-0000-4000-8000-000000000002	CONFLICTO	El precio cambio en el servidor
3	eeeeeeee-0000-4000-8000-000000000002	SIMULADO_APROBADO	Resultado de la comparacion …
4	33333333-0000-4000-8000-000000000003	RECHAZADO	Stock insuficiente en el servidor
5	55555555-0000-4000-8000-000000000004	RECHAZADO	Producto dado de baja en el servidor

Figura 7. Unidad 8: estados de sincronización tras comparar precios, stock y bajas con el servidor.

SQL 1*

```sql
SELECT
    p.nombre,
    i.cantidad AS cantidad_solicitada,
    s.stock AS stock_servidor
FROM item_borrador i
JOIN producto_cache p
    ON i.producto_id = p.producto_id
JOIN snapshot_servidor s
    ON i.producto_id = s.producto_id
WHERE i.id_local =
    'ffffffff-0000-4000-8000-000000000003';
```

	nombre	cantidad_solicitada	stock_servidor
1	Sprite 2L	10	4

Figura 8. Unidad 8: prueba de stock insuficiente (10 unidades solicitadas y 4 disponibles).

No cell active.
Type: NULL; Size: 0 bytes

Apply

SQL Log

Show SQL submitted by Error Log

Clear

```
(1555) abort at 50 in [INSERT INTO cola_sync (
    solicitud_uuid,
    id_local,
    estado
)
VALUES (
    '55555555-0000-4000-8000-000000000004',
    '44444444-0000-4000-8000-000000000004',
    'PENDIENTE'
```

SQL Log | Plot | DB Schema | Remote

Figura 9. Unidad 8: el UUID duplicado es rechazado por la clave primaria (código SQLite 1555).